

```
1 # Exercise 1: Handling ZeroDivisionError Imagine you have two numbers and want to determine what happens when you divide one number by the other.
2 # To do this, you need to create a Python function called safe_divide. You give this function two numbers, a 'numerator' and a 'denominator'.
3 # The 'numerator' is the number you want to divide, and the 'denominator' is the number you want to divide by.
4 # Use the input method of Python to take the values. The function should be able to do the division for you and give you the result.
5 # But here's the catch: if you try to divide by zero (which is not allowed in math),
6 # the function should be smart enough to catch that and tell you that it's not possible to divide by zero. Instead of showing an error,
7 # it should return None, which means 'nothing' or 'no value', and print "Error: Cannot divide by Zero."
8 # ans----
9 def safe_divide(numerator,denominator):
10     try:
11         result=numerator/denominator
12         return result
13     except ZeroDivisionError:
14         print("Error: Cannot divide by Zero")
15         return None
16 # taking the input from the user
17 try:
18     num_input = float(input("Enter the numerator: "))
19     den_input = float(input("Enter the denominator: "))
20
21     # Calling the function
22     division_result = safe_divide(num_input, den_input)
23     if division_result is not None:
24         print(f"The result is: {division_result}")
25 except ValueError:
26     print("Please enter valid numbers.")
```

```
vttha@Varun_thakur MINGW64 ~/OneDrive/Desktop/sem 3/python programming/assignments/5_ptatical 5 (main)
```

```
$ python pratical\ 5.py
```

```
Enter the numerator: 3
```

```
Enter the denominator: 0
```

```
Error: Cannot divide by Zero
```

```
vttha@Varun_thakur MINGW64 ~/OneDrive/Desktop/sem 3/python programming/assignments/5_ptatical 5 (main)
```

```
$
```

```

29 # =====Q2=====
30 # Exercise 2: Handling ValueError Imagine you have a number and want to calculate its square root. To do this,
31 # you need to create a Python function. You give this function one number, 'number1'.
32 # The function should generate the square root value if you provide a positive integer or float value as input.
33 # However, the function should be clever enough to detect the mistake if you enter a negative value.
34 # It should kindly inform you with a message saying, 'Invalid input! Please enter a positive integer or a float value.
35 import math
36
37 def calculate_square_root(number1):
38     try:
39         # math.sqrt will automatically raise a ValueError for negative numbers
40         result = math.sqrt(number1)
41         return result
42     except ValueError:
43         print("Invalid input! Please enter a positive integer or a float value.")
44         return None
45
46 # Example usage
47 try:
48     user_num = float(input("Enter a number to find its square root: "))
49     sqrt_result = calculate_square_root(user_num)
50     if sqrt_result is not None:
51         print(f"The square root is: {sqrt_result}")
52 except ValueError:
53     # This catches non-numeric text input from the user
54     print("Invalid input! Please enter a positive integer or a float value.")

```

Problems Output Debug Console Terminal Ports Spell Checker

bash - 5_ptatical 5

vttha@Varun thakur MINGW64 ~/OneDrive/Desktop/sem 3/python programming/assignments/5_ptatical 5 (main) ...

The result is: 0.0787037037037037

Enter a number to find its square root: -4

Invalid input! Please enter a positive integer or a float value.

```

56 # =====Q3=====
57 # Exercise 3: Handling Generic Exceptions Imagine you have a number and want to perform a complex mathematical task.
58 # The calculation requires dividing the value of the input argument "num" by the difference between "num" and 5,
59 # and the result has to be stored in a variable called "result". You have to define a function so that
60 # it can perform that complex mathematical task. The function should handle any potential errors that occur during the calculation.
61 # To do this, you can use a try-except block. If any exception arises during the calculation, it should
62 # the error using the generic exception class "Exception" as "e". When an exception occurs, the function should display
63 # "An error occurred during calculation."
64
65 def perform_complex_task(num):
66     try:
67         # This will raise a ZeroDivisionError if num is 5,
68         # or a TypeError if num is not a number.
69         result = num / (num - 5)
70         return result
71     except Exception as e:
72         print("An error occurred during calculation.")
73         return None
74
75 # Example usage
76 try:
77     user_value = float(input("Enter a number for the complex calculation: "))
78     task_result = perform_complex_task(user_value)
79     if task_result is not None:
80         print(f"The result is: {task_result}")
81 except ValueError:
82     print("Please enter a valid number.")

```

Problems Output Debug Console Terminal Ports Spell Checker

bash - 5_ptactical 5

```

vttha@Varun thakur MINGW64 ~/OneDrive/Desktop/sem 3/python programming/assignments/5_ptactical 5 (main) ...
Invalid input! Please enter a positive integer or a float value.
Enter a number for the complex calculation: 65
The result is: 1.0833333333333333

```